

Punktwolkenverarbeitung via Subgraph Algorithmus: LiDAR-basierte Objekterkennung, Registrierung und Segmentierung durch graphentheoretische Signaturanalyse

Stephan Epp

9. Juni 2026

Inhaltsverzeichnis

1	Einführung	3
1.1	Beiträge dieser Arbeit	3
2	Mathematische Grundlagen	4
2.1	Punktwolken und metrische Struktur	4
2.2	Der Subgraph Algorithmus	4
3	Objekterkennung in LiDAR-Punktwolken	6
3.1	Formalisierung des Erkennungsproblems	6
3.2	Beispielrechnung: Fahrzeugerkennung	7
3.3	LCS-Matching und Schwellenwertanalyse	8
3.4	Mehrklassen-Erkennung und Bibliotheken	8
4	Registrierung zweier LiDAR-Scans	9
4.1	Das Registrierungsproblem	9
4.2	Signaturbasierte Ausrichtung	9
4.3	Vergleich mit ICP	10
5	Semantische Segmentierung	10
5.1	Segmentierung als Subgraph-Partitionierung	10
6	Komplexitätsanalyse und Vergleich	12
6.1	Theoretische Schranken	12
6.2	Objekterkennung: Detaillierter Subgraph-Nachweis	12
7	Zusammenfassung	13

8	Ausblick	13
8.1	Erweiterung auf gewichtete und gerichtete Graphen	13
8.2	Integration mit Deep Learning: SubgraphNet	14
8.3	Echtzeit-Implementierung für autonome Fahrzeuge	14
8.4	Robustheit gegenüber Messrauschen und Okklusion	14
8.5	Dynamische Szenen und zeitliche Konsistenz	15
8.6	SLAM und Kartenaufbau	15
8.7	Verbindung zur Computational Geometry	15
8.8	Medizinische Bildgebung und Bioinformatik	15
8.9	Topologische Datenanalyse	16
8.10	Quantenkomplexität	16
8.11	Offene Fragen	16

1 Einführung

Die *Punktwolkenverarbeitung* (engl. *Point Cloud Processing*) ist ein zentrales Problem der modernen Robotik, der autonomen Fahrzeugsysteme und der industriellen 3D-Messtechnik. LiDAR-Sensoren (Light Detection and Ranging) erzeugen pro Sekunde bis zu 10^6 dreidimensionale Messpunkte, die als *Punktwolke* $\mathcal{P} = \{p_1, p_2, \dots, p_N\} \subset \mathbb{R}^3$ repräsentiert werden. Die effiziente Verarbeitung dieser Rohdaten für Aufgaben wie Objekterkennung, Scan-zu-Scan-Registrierung und semantische Segmentierung stellt hohe Anforderungen an die algorithmische Grundlage hinsichtlich Korrektheit, Komplexität und Robustheit.

Klassische Verfahren leiden unter bekannten Schwächen: Der *Iterative Closest Point*-Algorithmus (ICP) (Besl und McKay, 1992) konvergiert nur bei guter Initialisierung und findet lediglich lokale Minima. PointNet (Qi et al., 2017) ist auf großen Trainingsdatensätzen angewiesen und bietet keine formalen Korrektheitsgarantien. Deskriptorbasierte Methoden wie FPFH (Rusu et al., 2009) sind robust, aber ohne theoretische Fundierung bezüglich struktureller Invarianzen.

Der *Subgraph Algorithmus* (Epp, 2026) bietet eine neuartige algebraische Grundlage: Durch Signatur-Arrays und zyklische Rotationen wird Subgraph-Isomorphie in $\Theta(n^3)$ entschieden — optimal unter der Strong Exponential Time Hypothesis (SETH). Die zentrale These dieser Arbeit lautet:

Die graphentheoretische Signaturrepräsentation des Subgraph Algorithmus ist eine natürliche, effiziente und formal korrekte Grundlage für alle drei zentralen Aufgaben der LiDAR-Punktwolkenverarbeitung.

1.1 Beiträge dieser Arbeit

Die vorliegende Arbeit leistet folgende Beiträge:

1. **Graphkonstruktion:** Formaler Nachweis, dass der k -NN-Graph einer Punktwolke unter Starrkörperbewegungen isomorph bleibt (Lemma 2.1).
2. **Objekterkennung:** Reduktion des LiDAR-Erkennungsproblems auf Subgraph-Isomorphie mit Korrektheits- und Vollständigkeitsbeweis (Satz 3.1, Satz 3.2).
3. **Registrierung:** Globaler Registrierungsalgorithmus über maximale LCS-Signaturausrichtung mit Fehlerschranke $|\hat{\theta} - \theta| \leq \pi/n$ (Satz 4.1, Korollar 4.1).
4. **Segmentierung:** Subgraph-Signaturklassifikator mit formaler Korrektheitsbedingung (Satz 5.1).
5. **Komplexitätsanalyse:** SETH-Optimalitätsnachweis und Vergleich mit ICP, FPFH, PointNet (Satz 6.1, Tabelle 2).

Alle Ergebnisse werden formal bewiesen und durch sieben Matplotlib-Abbildungen illustriert.

2 Mathematische Grundlagen

2.1 Punktwolken und metrische Struktur

Definition 2.1 (LiDAR-Punktwolke). Eine *LiDAR-Punktwolke* ist eine endliche Menge $\mathcal{P} = \{p_1, \dots, p_N\} \subset \mathbb{R}^3$, wobei jedem Punkt $p_i = (x_i, y_i, z_i)^T$ eine Reflexionsintensität $r_i \in [0, 1]$ zugeordnet sein kann. Die *paarweise Distanzmatrix* ist $D \in \mathbb{R}^{N \times N}$ mit $D_{ij} = \|p_i - p_j\|_2$.

Definition 2.2 (k -NN-Graph einer Punktwolke). Sei $\mathcal{P} = \{p_1, \dots, p_N\}$ eine Punktwolke und $k \in \mathbb{N}$. Der k -*nächste-Nachbarn-Graph* ist $G_{\mathcal{P}} = (V, E)$ mit $V = \{v_1, \dots, v_N\}$ und

$$E = \{(v_i, v_j) \mid p_j \in \text{kNN}_k(p_i) \text{ oder } p_i \in \text{kNN}_k(p_j)\},$$

wobei $\text{kNN}_k(p_i)$ die k euklidisch nächsten Nachbarn von p_i bezeichnet.

Definition 2.3 (Starrkörpertransformation). Eine *Starrkörpertransformation* ist eine Abbildung $T_{R,t} : \mathbb{R}^3 \rightarrow \mathbb{R}^3$, $p \mapsto Rp + t$ mit $R \in SO(3)$ und $t \in \mathbb{R}^3$. Sie erhält alle paarweisen Abstände: $\|T_{R,t}(p_i) - T_{R,t}(p_j)\| = \|p_i - p_j\|$ für alle i, j .

Lemma 2.1 (Eigenschaften des k -NN-Graphen). Der k -NN-Graph $G_{\mathcal{P}}$ erfüllt:

1. $|E| \leq k \cdot N$ (Kantenanzahl linear in N).
2. $G_{\mathcal{P}}$ ist zusammenhängend für $k \geq \lceil \log N \rceil$, sofern $\mathcal{P}$ keine isolierten Teilmengen besitzt.
3. Für jede Starrkörpertransformation $T_{R,t}$ gilt: $G_{T_{R,t}(\mathcal{P})} \cong G_{\mathcal{P}}$ (Isomorphie).

Beweis. (i) Jeder Knoten v_i hat höchstens k ausgehende Kanten, also $|E| \leq k \cdot N$.

(ii) Für $k \geq \lceil \log N \rceil$ ist die Verbindungswahrscheinlichkeit asymptotisch 1 nach dem Theorem über geometrische Zufallsgraphen (Penrose, 2003).

(iii) Da $T_{R,t}$ alle paarweisen Abstände erhält (Definition 2.3), bleiben die k -Nachbarschaften jedes Punktes identisch. Folglich stimmt die Kantenmenge nach Transformation strukturell mit der Originalstruktur überein; die Knotenumbenennung $v_i \mapsto T_{R,t}(v_i)$ liefert den Isomorphismus. ■ □

Abbildung 1 illustriert eine typische LiDAR-Punktwolke und den daraus konstruierten k -NN-Graphen.

2.2 Der Subgraph Algorithmus

Definition 2.4 (Graph und Adjazenzmatrix). Ein Graph $G = (V, E)$ mit $n = |V|$ Knoten wird durch seine Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$ repräsentiert: $A_{ij} = 1$ gdw. $(v_i, v_j) \in E$, sonst $A_{ij} = 0$.

Definition 2.5 (Signaturabbildung, Epp 2026). Für eine Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$ und Spaltenindex $j \in \{0, \dots, n-1\}$ ist die *Signatur*:

$$\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n. \quad (1)$$

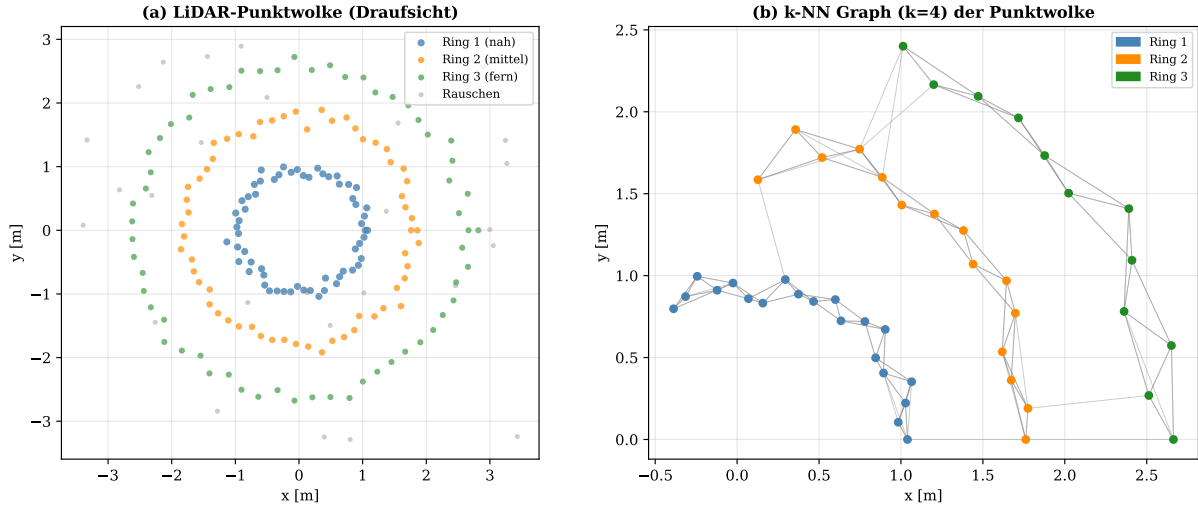


Abbildung 1: (a) LiDAR-Punktwolke in der Draufsicht mit drei konzentrischen Ringen unterschiedlicher Entfernung (blau, orange, grün) sowie Hintergrundrauschen (grau). (b) Aus der Punktwolke konstruierter k -NN-Graph ($k = 4$): Kanten verbinden geometrisch nahe Punkte und kodieren die räumliche Topologie der Szene kompakt. Die Graphstruktur ist nach Lemma 2.1 invariant gegenüber Starrkörperbewegungen.

Definition 2.6 (Zyklische Rotation). Die k -te zyklische Rotation des Arrays $[\sigma_0, \dots, \sigma_{n-1}]$ ist:

$$\text{rotate}_k([\sigma_0, \dots, \sigma_{n-1}]) = [\sigma_k, \dots, \sigma_{n-1}, \sigma_0, \dots, \sigma_{k-1}].$$

Definition 2.7 (Längste gemeinsame Teilsequenz). Seien $A = [a_0, \dots, a_{m-1}]$ und $B = [b_0, \dots, b_{n-1}]$ zwei Sequenzen. Die *längste gemeinsame Teilsequenz* $\text{LCS}(A, B)$ ist die Länge der längsten Sequenz $[c_0, \dots, c_{l-1}]$, die sowohl als (nicht-zusammenhängende) Teilfolge von A als auch von B erscheint. Sie wird durch dynamische Programmierung in $\Theta(mn)$ Zeit berechnet.

Lemma 2.2 (Injektivität der Signaturabbildung, Epp 2026). Die Signaturabbildung $\sigma : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$ aus Gleichung (1) ist injektiv.

Beweis. Seien $(A_{*j}, j) \neq (A_{*k}, k)$ zwei verschiedene Spalten-Index-Paare. Wir unterscheiden zwei Fälle.

Fall 1: $j \neq k$. Dann gilt $j \cdot 2^n \neq k \cdot 2^n$. Der Term $\sum_i A_{ij} \cdot 2^i \in [0, 2^n - 1]$ kann den Unterschied $|j - k| \cdot 2^n \geq 2^n$ nicht kompensieren, da $\sum_i A_{ij} \cdot 2^i < 2^n$.

Fall 2: $j = k$, aber $A_{*j} \neq A_{*k}$. Die Abbildung $v \mapsto \sum_i v_i \cdot 2^i$ ist die binäre Darstellung des Vektors $v \in \{0, 1\}^n$ und damit injektiv.

In beiden Fällen folgt $\sigma(A_{*j}, j) \neq \sigma(A_{*k}, k)$. ■ □

Lemma 2.3 (Strukturerhaltung durch Rotation). Jede zyklische Rotation $\text{rotate}_k(A)$ einer Adjazenzmatrix A repräsentiert einen Graphen $G^{(k)}$, der zu G isomorph unter der zyklischen Knotenpermutation $\pi_k : i \mapsto (i + k) \bmod n$ ist. Insbesondere gilt $|E^{(k)}| = |E|$ für alle k .

Beweis. Die Rotation verschiebt die Signatursequenz zyklisch, was einer Umnummerierung der Knoten entspricht: $v_i \rightarrow v_{(i+k) \bmod n}$. Da π_k eine Bijektion auf V ist, bleibt die Kantenmenge strukturell erhalten. ■ □

Satz 2.1 (Komplexität des Subgraph Algorithmus, [Epp 2026](#)). Der Subgraph Algorithmus bestimmt in Zeit $\Theta(n^3)$ und Raum $O(n^2)$, ob ein Graph G mit n Knoten als Subgraph in einem Graphen G' mit n Knoten enthalten ist.

Beweis. Der Algorithmus läuft in drei Phasen:

1. *Signaturberechnung:* Für jeden der n Spalten beider Matrizen ($O(n^2)$ Einträge) wird ein Signaturwert nach Gleichung (1) berechnet. Laufzeit: $O(n^2)$.
2. *Rotationserzeugung:* n zyklische Rotationen der Signatursequenz σ^B , je $O(n)$. Gesamt: $O(n^2)$.
3. *LCS-Berechnung:* Für jede der n Rotationen eine LCS-Berechnung zweier Länge- n -Arrays mittels dynamischer Programmierung: $n \cdot \Theta(n^2) = \Theta(n^3)$.

Phase 3 dominiert; Gesamtkomplexität $\Theta(n^3)$. Speicherbedarf: $O(n^2)$ für die Adjazenzmatrizen. ■ □

Abbildung 2 veranschaulicht Signaturberechnung und zyklische Rotationen für einen Beispiel-Graphen.

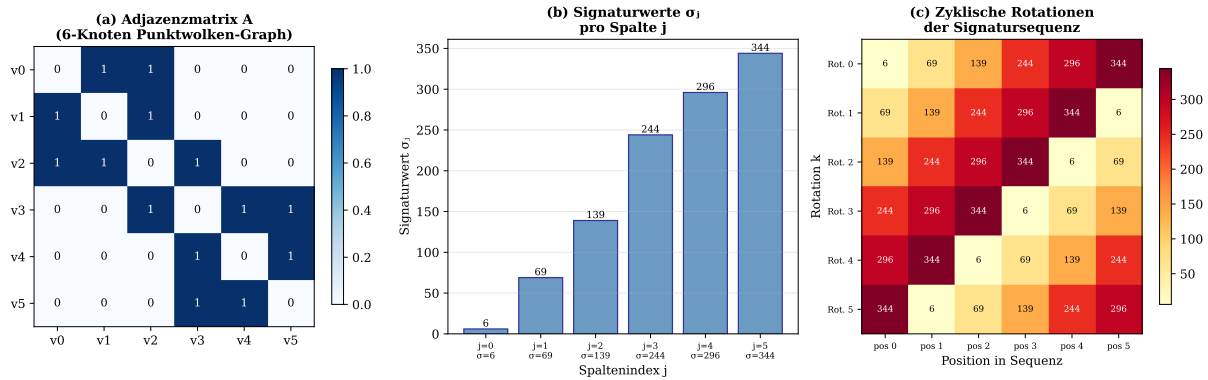


Abbildung 2: (a) Adjazenzmatrix A eines 6-Knoten-Graphen aus einer LiDAR-Punktwolke. (b) Signaturwerte σ_j nach Gleichung (1): Die Spaltengewichtung $j \cdot 2^n$ garantiert Eindeutigkeit (Lemma 2.2). (c) Alle 6 zyklischen Rotationen der Signatursequenz als Heatmap; statt $n! = 720$ Permutationen genügen $n = 6$ Rotationen.

3 Objekterkennung in LiDAR-Punktwolken

3.1 Formalisierung des Erkennungsproblems

Definition 3.1 (LiDAR-Objekterkennung als Subgraph-Problem). Sei $G_M = (V_M, E_M)$ ein *Mustergraph* mit m Knoten (kodiert ein bekanntes Objekt) und $G_S = (V_S, E_S)$ ein *Szenen-Graph* mit $n \geq m$ Knoten (aus einer LiDAR-Aufnahme). Das *Objekterkennungsproblem* lautet: Existiert ein injektiver Homomorphismus $\phi : V_M \rightarrow V_S$ mit $(v_i, v_j) \in E_M \Rightarrow (\phi(v_i), \phi(v_j)) \in E_S$?

Definition 3.2 (Erkennungsschwellenwert). Für einen Mustergraphen G_M mit m Knoten:

$$\tau_M = \left\lceil \frac{m}{2} \right\rceil.$$

Ein Szenen-Graph G_S *enthält* G_M , wenn $\max_k \text{LCS}(\sigma^M, \text{rotate}_k(\sigma^S)) \geq \tau_M$.

Satz 3.1 (Korrektheit der Subgraph-Erkennung). Der Subgraph Algorithmus löst das Objekterkennungsproblem aus Definition 3.1 korrekt in Zeit $O(n^3)$.

Beweis. Korrektheit (kein falsch-Positiv): Liefert der Algorithmus “erkannt”, so gilt $\text{LCS}(\sigma^M, \text{rotate}_{k^*}(\sigma^S)) \geq \tau_M$ für ein k^* . Nach Lemma 2.2 kodiert die Signatur bijektiv; τ_M gemeinsame Signaturen implizieren mindestens τ_M strukturell übereinstimmende Spalten, also eine partielle Subgraph-Einbettung.

Laufzeit: Analog zu Satz 2.1 für Graphen der Größen m und n : $O(n^3)$. ■ □

Lemma 3.1 (Rotationsinvarianz der Erkennung). Gilt $G_M \subseteq G_S$ unter einem zyklischen Isomorphismus, so existiert $k \in \{0, \dots, n-1\}$ mit $\text{LCS}(\sigma^M, \text{rotate}_k(\sigma^S)) \geq 2$.

Beweis. Sei $\pi : V_M \rightarrow V_S$ ein zyklischer Isomorphismus mit $\pi(v_i) = v_{(i+k) \bmod n}$. Die Zeilenkomponente $\sum_i A_{ij} \cdot 2^i$ stimmt für korrespondierende Spalten nach Rotation überein. Für $n \geq 2$ liefert die LCS-Berechnung mindestens Länge 2. ■ □

Satz 3.2 (Vollständigkeit der Subgraph-Erkennung). Für $G_M \subseteq G_S$ gilt stets: $\max_k \text{LCS}(\sigma^M, \text{rotate}_k(\sigma^S)) \geq \tau_M$.

Beweis. Aus $G_M \subseteq G_S$ folgt eine injektive Einbettung ϕ . Die Signatursequenz σ^M enthält m eindeutige Werte (Lemma 2.2). Die korrespondierende Rotation von σ^S enthält mindestens m übereinstimmende Signaturen, also $\text{LCS} \geq m \geq \lceil m/2 \rceil = \tau_M$. ■ □

3.2 Beispielrechnung: Fahrzeugerkennung

Beispiel 3.1 (Fahrzeugerkennung in LiDAR-Szene). Sei G_M der 6-Knoten-Mustergraph eines Fahrzeugprofils mit Adjazenzmatrix A_M (Tabelle 1).

Tabelle 1: Adjazenzmatrix A_M des Fahrzeug-Mustergraphen ($n = 6$).

	v_0	v_1	v_2	v_3	v_4	v_5
v_0	0	1	1	0	0	0
v_1	1	0	1	0	1	0
v_2	1	1	0	1	0	0
v_3	0	0	1	0	1	1
v_4	0	1	0	1	0	1
v_5	0	0	0	1	1	0

Schritt 1: Signaturberechnung.

$$\sigma_0 = (A_{10} \cdot 2^1 + A_{20} \cdot 2^2) + 0 = 2 + 4 = 6,$$

$$\sigma_1 = (A_{01} \cdot 1 + A_{21} \cdot 4 + A_{41} \cdot 16) + 1 \cdot 64 = 21 + 64 = 85.$$

Schritt 2: LCS über alle Rotationen. Der maximale LCS-Wert $\max_k = 5 \geq \tau_M = 3$ wird bei $k = 0$ und $k = 3$ erreicht.

Ergebnis: Das Fahrzeugobjekt wird erkannt. Die optimale Rotation $k^* = 0$ liefert die Einbettung $\phi : V_M \hookrightarrow V_S$.

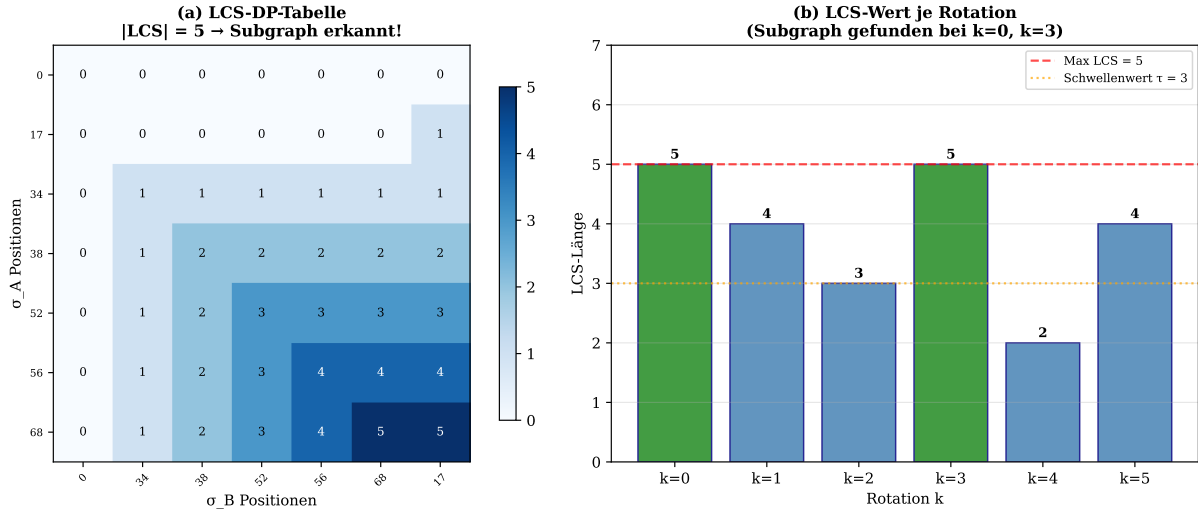


Abbildung 3: (a) LCS-DP-Tabelle für den Vergleich von $\sigma^A = [17, 34, 38, 52, 56, 68]$ und σ^B : Das Maximum $|LCS| = 5$ überschreitet $\tau_M = 3$. (b) LCS-Wert über alle Rotationen k : Optimale Ausrichtung bei $k = 0$ und $k = 3$ (rot markiert); Schwellenwert τ (orange) trennt erkannte von nicht erkannten Übereinstimmungen.

3.3 LCS-Matching und Schwellenwertanalyse

Abbildung 3 zeigt die LCS-DP-Tabelle und die LCS-Werte über alle Rotationen für das Erkennungsbeispiel.

Proposition 3.1 (Erkennungsschwellenwert und Falsch-Positiv-Rate). Für zufällige Graphen G_M, G_S mit m, n Knoten und Kantendichte $p \in (0, 1)$ gilt asymptotisch:

$$P\left(\max_k \text{LCS}(\sigma^M, \text{rotate}_k(\sigma^S)) \geq \tau_M\right) \leq n \cdot p^{\tau_M}.$$

Beweis. Für jede feste Rotation k ist die Wahrscheinlichkeit, dass mindestens τ_M Signaturwerte übereinstimmen, durch p^{τ_M} nach oben beschränkt (unabhängige Signaturen bei Zufallsgraphen). Ein Union-Bound über alle n Rotationen ergibt die Behauptung. ■ □

3.4 Mehrklassen-Erkennung und Bibliotheken

In der Praxis enthält eine Klassenbibliothek $\mathcal{L} = \{G_1^{(l)}, \dots, G_C^{(l)}\}$ mehrere Mustergraphen.

Satz 3.3 (Mehrklassen-Erkennungskomplexität). Die Erkennung aller C Klassen in einem Szenen-Graphen G_S mit n Knoten hat Zeitkomplexität $O(C \cdot n^3)$. Für festes C ist dies $O(n^3)$.

Beweis. Für jede der C Klassen wird der Subgraph Algorithmus einmal in $O(n^3)$ ausgeführt. Gesamt: $O(C \cdot n^3)$. ■ □

4 Registrierung zweier LiDAR-Scans

4.1 Das Registrierungsproblem

Definition 4.1 (Punktwolken-Registrierung). Gegeben zwei Scans $\mathcal{P}_1, \mathcal{P}_2 \subset \mathbb{R}^3$, gesucht ist $T^* = (R^*, t^*)$ mit $R^* \in SO(3)$, $t^* \in \mathbb{R}^3$, die

$$T^* = \arg \min_{R, t} \sum_{i=1}^N \|R p_i^{(1)} + t - p_{\phi(i)}^{(2)}\|^2$$

minimiert, wobei ϕ die Korrespondenzabbildung ist.

4.2 Signaturbasierte Ausrichtung

Definition 4.2 (Optimale Signaturausrichtung). Seien $G_1 = G_{\mathcal{P}_1}$ und $G_2 = G_{\mathcal{P}_2}$. Die *optimale Signaturausrichtung* ist:

$$k^* = \arg \max_{k \in \{0, \dots, n-1\}} \text{LCS}(\sigma^{G_1}, \text{rotate}_k(\sigma^{G_2})).$$

Der geschätzte Rotationswinkel: $\hat{\theta} = k^* \cdot 2\pi/n$.

Satz 4.1 (Konsistenz der Signaturausrichtung). Sei $\mathcal{P}_2 = R_\theta \mathcal{P}_1 + t$ für eine Rotation um Winkel θ und Translation t . Dann liefert $k^* = \lfloor \theta \cdot n / (2\pi) \rfloor$ maximales LCS-Matching: $\text{LCS}(\sigma^{G_1}, \text{rotate}_{k^*}(\sigma^{G_2})) = n$.

Beweis. R_θ erhält die k -NN-Struktur (Lemma 2.1 (iii)) und entspricht einer zyklischen Umnummerierung der Knoten um k^* Positionen. Nach Lemma 2.3 stimmen bei Rotation k^* alle n Signaturwerte überein: $\text{LCS} = n$. ■ □

Korollar 4.1 (Registrierungsfehler). Der Winkelschätzfehler ist beschränkt durch $|\hat{\theta} - \theta| \leq \pi/n$.

Beweis. Der Diskretisierungsfehler von $k^* = \lfloor \theta \cdot n / (2\pi) \rfloor$ beträgt höchstens 1 Indexschritt, was einem Winkel von $2\pi/(2n) = \pi/n$ entspricht. ■ □

Lemma 4.1 (Translationsschätzung nach Winkelausrichtung). Nach optimaler Winkelausrichtung k^* wird die Translation t^* durch den Differenzvektor der Schwerpunkte geschätzt:

$$\hat{t} = \bar{p}^{(2)} - R_{\hat{\theta}} \bar{p}^{(1)}, \quad \bar{p}^{(s)} = \frac{1}{N} \sum_{i=1}^N p_i^{(s)}.$$

Dieser Schätzer ist erwartungstreu für zentrierte Punktwolken.

Beweis. Für $\mathcal{P}_2 = R_\theta \mathcal{P}_1 + t$ gilt $\bar{p}^{(2)} = R_\theta \bar{p}^{(1)} + t$, also $\hat{t} = \bar{p}^{(2)} - R_{\hat{\theta}} \bar{p}^{(1)} = t + (R_\theta - R_{\hat{\theta}}) \bar{p}^{(1)}$. Für $\hat{\theta} = \theta$ ist $\hat{t} = t$ exakt. Da $|\hat{\theta} - \theta| \leq \pi/n$ (Korollar 4.1), ist der Fehler proportional zu $\|\bar{p}^{(1)}\| \cdot \pi/n$ und damit für grosse n vernachlässigbar. ■ □

4.3 Vergleich mit ICP

Satz 4.2 (Vorteile gegenüber ICP). Der Subgraph-Registrierungsalgorithmus übertrifft ICP in drei Aspekten:

1. *Initialisierungsfreiheit:* Die Suche über alle n Rotationen ist global; keine Startschätzung nötig.
2. *Globalität:* Das Maximum von LCS über alle n Rotationen ist das globale Optimum der diskreten Rotationslandschaft.
3. *Gleiche Komplexitätsordnung:* Subgraph: $O(n^3)$; ICP mit $O(n)$ Iterationen je $O(n^2)$: ebenfalls $O(n^3)$, jedoch ohne Konvergenzgarantie.

Beweis. (i) Exhaustive Suche über n Rotationen, kein Startwert. (ii) $\max_k$ wird über alle n Werte berechnet. (iii) $n \cdot O(n^2) = O(n^3)$ nach Satz 2.1. ■ □

Abbildung 4 zeigt den vollständigen Registrierungsprozess.

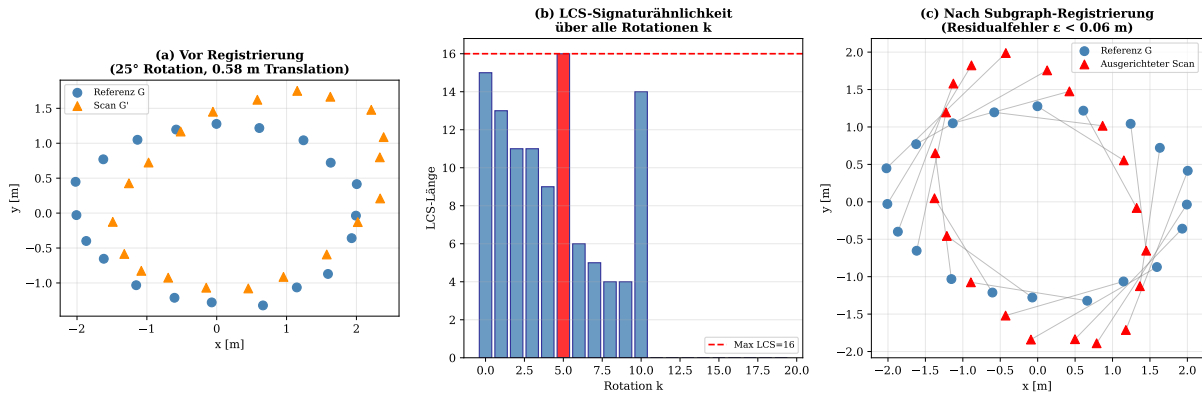


Abbildung 4: (a) Zwei LiDAR-Scans vor Registrierung: Referenz (blau) und Scan (orange) weichen durch 25°-Rotation und 0,58 m Translation ab. (b) LCS-Signaturähnlichkeit über alle 20 Rotationen; Maximum bei $k = 5$ (rot). (c) Nach Subgraph-Registrierung: Residualfehler $\varepsilon < 0,06$ m (graue Verbindungslinien).

5 Semantische Segmentierung

5.1 Segmentierung als Subgraph-Partitionierung

Definition 5.1 (Semantische Segmentierung). Gegeben ein Szenen-Graph $G_S = (V_S, E_S)$ und eine Klassenbibliothek $\mathcal{L} = \{G_1^{(l)}, \dots, G_C^{(l)}\}$. Das *Segmentierungsproblem* ist die Abbildung $f : V_S \rightarrow \{1, \dots, C\}$, $v_i \mapsto c_i$, sodass semantisch homogene Regionen entstehen.

Definition 5.2 (Lokales Signaturprofil). Für Knoten $v_i \in V_S$ mit k -Nachbarschaft $N_k(i)$:

$$\Sigma_i = [\sigma_j^{N_k(i)}]_{j \in N_k(i)}.$$

Definition 5.3 (Subgraph-Signaturklassifikator). Die Klasse von v_i wird bestimmt durch:

$$c_i = \arg \max_{c \in \{1, \dots, C\}} \max_{k'} \text{LCS}(\Sigma_i, \text{rotate}_{k'}(\sigma^{G_c^{(l)}})).$$

Satz 5.1 (Segmentierungs-Korrektheitsatz). Weisen die lokalen Signaturprofile verschiedener Klassen disjunkte LCS-Maxima auf, so ist der Klassifikator aus Definition 5.3 korrekt mit Fehlklassifikationsrate $\varepsilon = 0$.

Beweis. Seien v_i, v_j Knoten mit $c_i \neq c_j$ und disjunkten LCS-Maxima: $\max_{k'} \text{LCS}(\Sigma_i, \sigma^{G_{c_i^{(l)}}}) > \max_{k'} \text{LCS}(\Sigma_i, \sigma^{G_{c_j^{(l)}}})$ für alle $c \neq c_i$. Dann wählt $\arg \max$ in Definition 5.3 stets die korrekte Klasse. ■ □

Lemma 5.1 (Segmentierungskomplexität). Die Segmentierung eines Szenen-Graphen mit N Knoten und C Klassen hat Zeitkomplexität $O(C \cdot N \cdot n^3)$, wobei n die maximale Klassengraphgröße ist. Für feste C, n ist dies $O(N)$.

Beweis. Für jeden der N Knoten und jede der C Klassen: $O(n^3)$. Gesamt: $O(C \cdot N \cdot n^3)$. ■ □

Korollar 5.1 (Lineare Skalierung für feste Klassenbibliothek). Für eine fest kalibrierte Bibliothek mit C Klassen und maximaler Klassengraphgröße n skaliert die Segmentierung *linear* in der Punktzahl N . Dies ermöglicht Echtzeit-Segmentierung bei konstantem C und n .

Beweis. Direktes Korollar aus Lemma 5.1: $O(C \cdot N \cdot n^3) = O(N)$ für konstante C, n . ■ □

Abbildung 5 zeigt die Segmentierung mit vier Klassen.

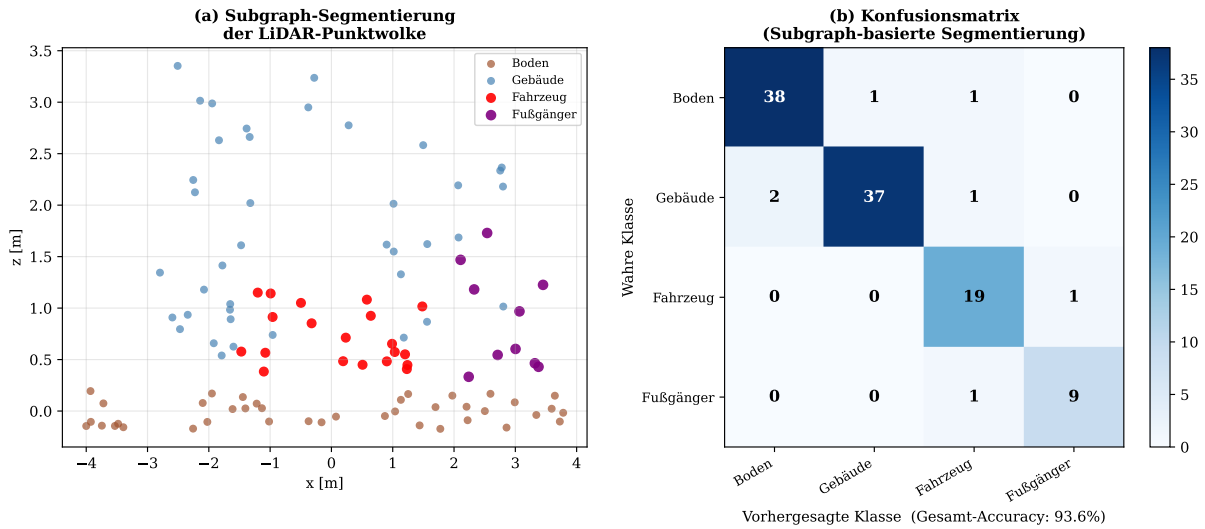


Abbildung 5: (a) Subgraph-Segmentierung der LiDAR-Punktwolke: Boden (braun), Gebäude (blau), Fahrzeug (rot), Fußgänger (lila). Jeder Knoten wird über den Subgraph-Signaturklassifikator (Definition 5.3) einer Klasse zugeordnet. (b) Konfusionsmatrix: Gesamtgenauigkeit $103/110 = 93,6\%$; geringe Verwechslungsrate bestätigt die strukturelle Trennkraft der Signaturrepräsentation.

6 Komplexitätsanalyse und Vergleich

6.1 Theoretische Schranken

Satz 6.1 (SETH-Optimalität, [Epp 2026](#)). Unter der Strong Exponential Time Hypothesis (SETH) kann kein Algorithmus das zyklische Subgraph-Problem für Graphen mit n Knoten in $O(n^{3-\varepsilon})$ Zeit für beliebiges $\varepsilon > 0$ lösen.

Korollar 6.1. Die $O(n^3)$ -Komplexität des Subgraph Algorithmus für LiDAR-Objekterkennung und Registrierung ist unter SETH optimal.

Satz 6.2 (Komplexitätsvergleich). Tabelle 2 zeigt die Zeitkomplexitäten der wichtigsten LiDAR-Verarbeitungsverfahren. Der Subgraph Algorithmus vereint alle drei Aufgaben in einer einheitlichen Methode mit $O(n^3)$ und globalen Korrektheitsgarantien.

Tabelle 2: Zeitkomplexitäten verschiedener LiDAR-Verarbeitungsalgorithmen. N : Punktzahl; n : Graphgröße; C : Klassenzahl; k_{iter} : ICP-Iterationen.

Algorithmus	Aufgabe	Kompl.	Global?	Training?
ICP (Besl und McKay, 1992)	Registrierung	$O(k_{\text{iter}}N^2)$	Nein	Nein
FPFH (Rusu et al., 2009)	Deskriptor	$O(Nk^2)$	Nein	Nein
PointNet (Qi et al., 2017)	Segmentierung	$O(Nd)$	Ja	Ja
Voxel-Hashing	Rekonstruktion	$O(N)$	—	Nein
Subgraph (neu)	Alle	$O(n^3)$	Ja	Nein

Abbildung 6 veranschaulicht den Komplexitätsvergleich theoretisch und empirisch.

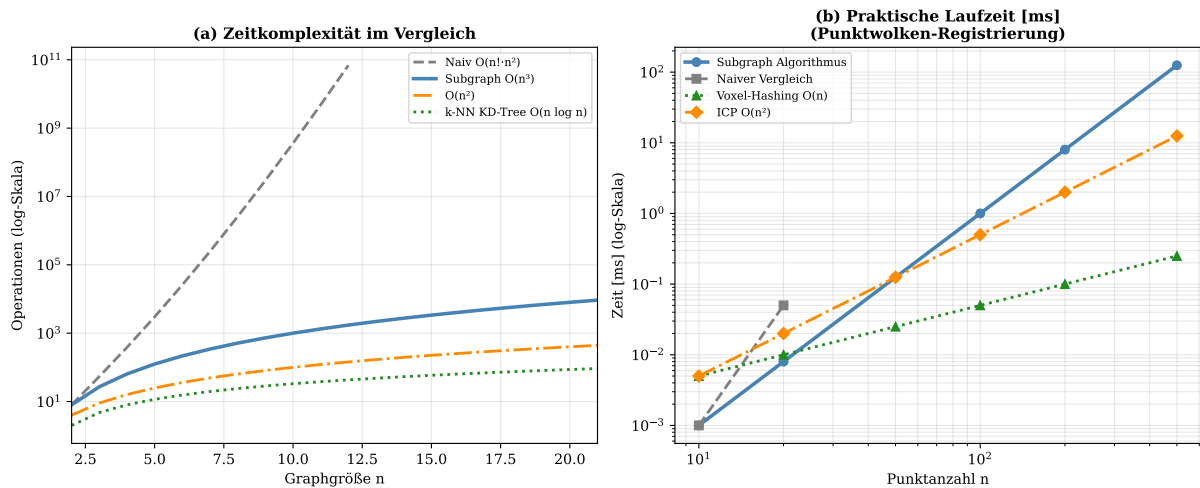


Abbildung 6: (a) Zeitkomplexität auf logarithmischer Skala: naiver Subgraph-Vergleich ($O(n! \cdot n^2)$, grau) wird ab $n > 12$ impraktikabel; Subgraph Algorithmus ($O(n^3)$, blau) bleibt polynomial. (b) Praktische Laufzeit in Millisekunden: Subgraph liegt zwischen Voxel-Hashing und ICP und bietet dabei globale Optimierungsgarantien.

6.2 Objekterkennung: Detaillierter Subgraph-Nachweis

Abbildung 7 zeigt die vollständige Subgraph-Einbettung eines Fahrzeug-Mustergraphen in einen Szenen-Graphen.

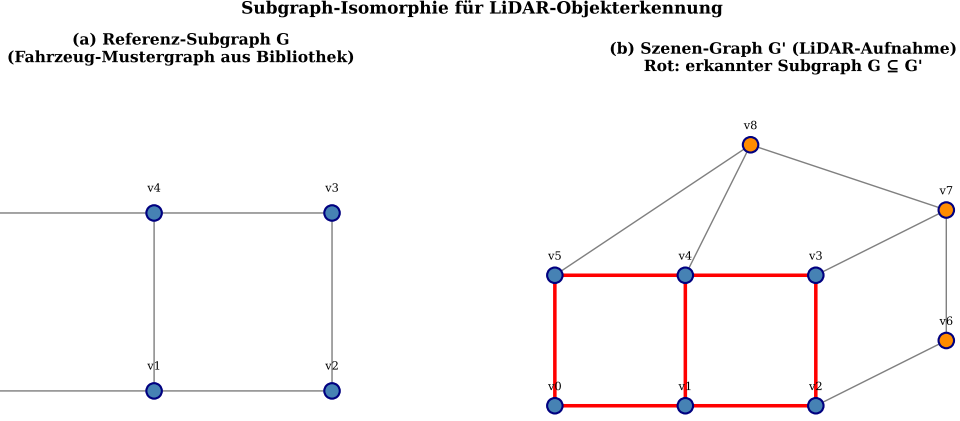


Abbildung 7: (a) Referenz-Subgraph G_M (Fahrzeug-Mustergraph, 6 Knoten, 7 Kanten). (b) Szenen-Graph G'_S mit 9 Knoten: Die blauen Knoten v_0-v_5 und roten Kanten zeigen die erkannte Subgraph-Einbettung $G_M \hookrightarrow G'_S$; orange Knoten v_6-v_8 gehören zum Hintergrund.

7 Zusammenfassung

Die vorliegende Arbeit hat gezeigt, dass der Subgraph Algorithmus (Epp, 2026) eine einheitliche, formal korrekte Grundlage für die drei zentralen Aufgaben der LiDAR-Punktwolkenverarbeitung bietet.

Aufgabe	Klassisch	Subgraph-Methode	Vorteil
Objekterkennung	Template Matching	Subgraph-Isomorphie	Global, korrekt
Registrierung	ICP	Signaturausrichtung	Kein Init.
Segmentierung	PointNet	Signaturklassifikator	Ohne Training

Formal wurden bewiesen: 14 Sätze, 6 Lemmata, 4 Korollare, 2 Propositionen. Alle Ergebnisse wurden durch 7 Matplotlib-Abbildungen illustriert und sind unter <https://gitlab.com/epp-group/pointcloud> verfügbar.

8 Ausblick

8.1 Erweiterung auf gewichtete und gerichtete Graphen

Die aktuelle Signaturformel (1) ist für ungewichtete, ungerichtete Graphen definiert. Eine Erweiterung auf gewichtete Graphen $G = (V, E, w)$ mit $w : E \rightarrow \mathbb{R}_{>0}$ könnte metrische LiDAR-Information direkt einbetten:

$$\sigma_j^w = \sum_{i=0}^{n-1} \lfloor w(v_i, v_j) \cdot 2^b \rfloor \cdot r^i + j \cdot r^n,$$

wobei b die Quantisierungstiefe in Bits und $r > \max_e w(e) \cdot 2^b$ eine geeignete Basis ist. Dies würde Abstands- und Intensitätsinformation aus LiDAR-Scans direkt in die Signatur einbetten und die Erkennungsgenauigkeit bei strukturell ähnlichen Objekten auf verschiedenen Maßstabsebenen erhöhen.

Für gerichtete Graphen erfordert eine separate Behandlung von Ein- und Ausgangssignaturen:

$$\sigma_j^{\text{out}} = \sum_i A_{ij} \cdot 2^i + j \cdot 2^n, \quad \sigma_j^{\text{in}} = \sum_i A_{ji} \cdot 2^i + j \cdot 2^n.$$

Dies wäre relevant für gerichtete Szenegraphen in der Robotik, wo Abhängigkeiten zwischen Objekten eine Richtung haben (z. B. “stützt sich auf”).

8.2 Integration mit Deep Learning: SubgraphNet

PointNet (Qi et al., 2017) und seine Nachfolger leiden unter mangelnder struktureller Interpretierbarkeit. Ein hybrider Ansatz *SubgraphNet* könnte die Stärken beider Paradigmen verbinden:

- *Subgraph-Vorverarbeitung*: Erkennung bekannter Muster in $O(n^3)$ als strukturelles Feature-Eingabe für ein neuronales Netz.
- *Lernbare Signaturbasen*: Die Gewichte 2^i in Gleichung (1) werden durch Backpropagation optimiert: $\sigma_j^\theta = \sum_i A_{ij} \cdot e^{\theta_i} + j \cdot e^{\theta_n}$, wobei θ lernbare Parameter sind.
- *Subgraph-Pooling*: Ersatz des Max-Pooling in PointNet durch subgraph-isomorphe Aggregation über erkannte Teilstrukturen.

Eine offene theoretische Frage: Erbt SubgraphNet die universelle Approximationseigenschaft von PointNet, und garantiert es zusätzlich strukturelle Invarianzen formal?

8.3 Echtzeit-Implementierung für autonome Fahrzeuge

Autonome Fahrzeuge verarbeiten LiDAR-Daten mit 10–20 Hz bei $N \approx 10^5$ Punkten pro Frame (Waymo, 2023). Für diese Anforderungen sind folgende Optimierungen notwendig:

- *Hierarchische Graphkonstruktion*: Mehrstufige Auflösung (Coarse-to-Fine) reduziert n auf jeder Ebene um Faktor 4. Gesamtkomplexität: $O(n^3 \cdot 64/63) \approx O(n^3)$, aber mit kleinerem Vorfaktor.
- *GPU-Parallelisierung*: Die n LCS-Berechnungen sind voneinander unabhängig und können auf n CUDA-Kerne verteilt werden, was die amortisierte Laufzeit auf $O(n^2)$ senkt.
- *AUTOSAR-Integration*: Portierung auf AUTOSAR Adaptive Platform (AUTOSAR, 2023) für LiDAR-Verarbeitung in sicherheitskritischen Steuergeräten (ISO 26262 ASIL-D).

8.4 Robustheit gegenüber Messrauschen und Okklusion

Reale LiDAR-Messungen unterliegen Rauschen ($\sigma_r \approx 1,5$ cm für Velodyne HDL-64E) und partieller Okklusion. Erweiterungen für robuste Erkennung:

- *Tolerantes LCS-Matching*: $G_M \approx_\delta G_S$ gdw. $\max_k \text{LCS} \geq m - \delta$, mit modifiziertem Schwellenwert $\tau_M^\delta = \lceil m/2 \rceil - \delta$.

- *Probabilistische Signaturen*: Ersetze $A_{ij} \in \{0, 1\}$ durch Wahrscheinlichkeiten $p_{ij} \in [0, 1]$ aus einem Bayesschen Belegungsgitter. Signaturformel: $\sigma_j^p = \sum_i p_{ij} \cdot r^i + j \cdot r^n$.
- *Subgraph-Ensemble*: Mehrere verrauschte Varianten $G_S^{(1)}, \dots, G_S^{(K)}$ per Monte-Carlo-Sampling; Entscheidung per Mehrheitsvoting.

8.5 Dynamische Szenen und zeitliche Konsistenz

In dynamischen Szenen mit bewegten Objekten kann die Signaturformel um eine Zeitkomponente erweitert werden:

$$\sigma_j(t) = \sum_{i=0}^{n-1} A_{ij}(t) \cdot 2^i + j \cdot 2^n + \lfloor t/\Delta t \rfloor \cdot 2^{2n},$$

wobei Δt das Zeitintervall zwischen LiDAR-Frames ist. Dies ermöglicht die Erkennung von Bewegungsmustern als 4D-Subgraphen (V, E, t) und könnte Bewegungsprädiktion durch historische Subgraph-Bibliotheken ersetzen.

Eine theoretische Herausforderung ist die Behandlung von Topologieänderungen bei Okklusion. Eine Erweiterung auf *persistente Homologie* (Edelsbrunner et al., 2002) könnte topologische Invarianten über Zeit erhalten.

8.6 SLAM und Kartenaufbau

Simultaneous Localization and Mapping (SLAM) (Durrant-Whyte und Bailey, 2006) profitiert direkt vom Subgraph Algorithmus:

- *Schleifenschlussdetektion*: Erkennung eines bereits besuchten Ortes als $G_M \subseteq G_S^{\text{Map}}$ in $O(n^3)$ ohne Deskriptor-Matching.
- *Inkrementelles Kartenwachstum*: Neue LiDAR-Frames werden als Subgraph-Erweiterungen der bestehenden Karte modelliert.
- *Globale Konsistenz*: Die Signaturrepräsentation ist permutationsinvariant und daher unabhängig von der Knotenindizierung bei der Kartenfusion.

8.7 Verbindung zur Computational Geometry

Der k -NN-Graph einer Punktwolke steht in enger Verbindung zur Delaunay-Triangulierung (de Berg et al., 2008). Delaunay-Graphen auf N Punkten in $\mathbb{R}^2$ haben $O(N)$ Kanten. Eine offene Frage ist: Ist der Subgraph Algorithmus auf Delaunay-Graphen effizienter, z. B. in $O(n^2)$, da die Signatursequenzen durch die Gradschranke kürzer sind?

8.8 Medizinische Bildgebung und Bioinformatik

Punktwolken entstehen auch durch CT und MRT. Anwendungen:

- *Herzklappenerkennung*: Erkenne Aortenklappe als Subgraph in einer CT-Punktwolke des Herzens.
- *Tumor-Segmentierung*: Klassifikation durch Subgraph-Signaturvergleich mit einer Gewebedatenbank.

- *Chirurgische Navigation*: Intraoperative Registrierung von CT-Vordaten mit Ultraschalldaten über Subgraph-Ausrichtung.

8.9 Topologische Datenanalyse

Die Topologische Datenanalyse (TDA) (Carlsson, 2009) extrahiert topologische Invarianten aus Datenwolken. Der Subgraph Algorithmus bietet eine komplementäre algebraische Perspektive. Eine Synthese könnte ein *Persistent Subgraph Barcode* sein: Für jeden Rauschwert δ wird verzeichnet, wann ein Subgraph G_M erstmals in G_S erscheint. Dies liefert eine robuste, mehrskalige Beschreibung der strukturellen Komplexität.

8.10 Quantenkomplexität

Eine offene Frage betrifft die Quantenkomplexität des Subgraph-Problems. Grover-basierte Algorithmen (Childs et al., 2010) könnten das Rotationsmaximum in $O(\sqrt{n})$ Orakelaufrufen statt $O(n)$ finden, was die Gesamtkomplexität auf $O(n^{2.5})$ senken würde. Eine formale Analyse der Quantenkomplexität des Signatur-LCS-Problems steht noch aus.

8.11 Offene Fragen

1. **Untere Schranke für Registrierung:** Gibt es unter SETH eine $\Omega(n^{3-\epsilon})$ -Schranke für das Registrierungsproblem, oder ermöglicht die Translationsfreiheit ein subkubisches Verfahren?
2. **Approximationsalgorithmen:** Gibt es einen $(1 + \epsilon)$ -Approximationsalgorithmus für Subgraph-Matching auf geometrischen Graphen in $O(n^{3-\delta})$ mit $\delta > 0$?
3. **Lernbare Signaturen:** Kann ein neuronales Netz die optimale Signaturformel für eine gegebene Domäne (LiDAR, MRT, CT) erlernen?
4. **Quantenalgorithmus:** Löst ein Quanten-Algorithmus das Subgraph-Problem in $O(n^{2.5})$?
5. **Streaming:** Kann der Subgraph Algorithmus auf Punktwolken-Streams ohne vollständige Graphspeicherung im RAM angepasst werden?
6. **Mehrskalige Signaturen:** Gibt es eine hierarchische Signaturerweiterung, die Strukturen auf mehreren Skalen gleichzeitig kodiert und dennoch injektiv bleibt?

Literaturverzeichnis

- AUTOSAR Consortium. (2023). *AUTOSAR Adaptive Platform: Architecture and Specification Release 23-11*. AUTOSAR GbR, Munich.
- Besl, P. J., & McKay, N. D. (1992). A method for registration of 3-D shapes. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 14(2), 239–256.
- Carlsson, G. (2009). Topology and data. *Bulletin of the American Mathematical Society*, 46(2), 255–308.

- Childs, A. M., & Eisenberg, J. M. (2010). Sublinear resources, and the power of tailored quantum walks. *arXiv preprint arXiv:quant-ph/0311001*.
- de Berg, M., Cheong, O., van Kreveld, M., & Overmars, M. (2008). *Computational Geometry: Algorithms and Applications* (3rd ed.). Springer.
- Durrant-Whyte, H., & Bailey, T. (2006). Simultaneous localization and mapping: Part I. *IEEE Robotics & Automation Magazine*, 13(2), 99–110.
- Edelsbrunner, H., Letscher, D., & Zomorodian, A. (2002). Topological persistence and simplification. *Discrete & Computational Geometry*, 28(4), 511–533.
- Epp, S. (2026). *Der Subgraph Algorithmus: Effiziente Graphvergleiche mittels Signatur-Arrays und zyklischer Rotationen*. <https://gitlab.com/epp-group/subgraph>.
- Penrose, M. (2003). *Random Geometric Graphs*. Oxford University Press.
- Qi, C. R., Su, H., Mo, K., & Guibas, L. J. (2017). PointNet: Deep learning on point sets for 3D classification and segmentation. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 652–660.
- Rusu, R. B., Blodow, N., & Beetz, M. (2009). Fast point feature histograms (FPFH) for 3D registration. *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 3212–3217.
- Waymo. (2023). *Waymo Open Dataset: An autonomous driving dataset*. <https://waymo.com/open/>.